

Projet Designs Patterns en C#

Table des matières

1 Informations pratiques	3
1.1 Rendus attendus	3
1.2 Utilisation de git	3
1.3 Utilisation de l'IA générative	3
2.1 Implémentation naïve d'une solution	4
2.2 Mise en place de design patterns	4
2.3 Implémentation de modules supplémentaires « libres »	4
2.4 Soutenance	5
3 Sujet	6
3.1 Prise en compte des commandes	6
3.2 Listing des instructions	7
3.2.1 Edition de l'inventaire des pièces disponibles	7
3.2.2 Edition de l'inventaire des pièces nécessaires	7
3.2.3 Édition des instructions d'assemblage	8
3.2.4 Vérification d'une commande	9
3.2.5 Production d'une commande	9
3.3 Notations spécifiques	10
3.4 Comportement attendu du programme	10
4 Première extension de projet	11
4.1 Mise en place de design patterns	11
4.2 Différents types de drones	11
4.3 Ajout de templates de drones	12
5 Modules complémentaires	13
5.1 Consignes générales	13
5.1.1 Recevoir du stock	13
5.1.2 Contraintes des constructions de drones	13
5.2 Détails des nouvelles instructions disponibles	13
5.2.1 Modification de drones	13
5.2.2 Gestion de commandes	14
5.2.3 Traçabilité des flux	15
5.2.4 Gestion d'un groupe d'usines	15

6 Récapitulatifs et données	17
6.1 Récapitulatif des instructions disponibles pour l'utilisateur.....	17
6.2 Pièces et drones disponibles	17
7 Exemples.....	19
7.1 Demande d'instructions valide.....	19
7.2 Vérification d'une commande invalide.....	19

1 Informations pratiques

1.1 Rendus attendus

Le code rendu devra compiler sans erreur.

Mieux vaut rendre un projet incomplet qui compile qu'un code ne compilant pas.

Le projet sera noté selon plusieurs critères :

- qualité du code fourni
- prise en compte des différents retours
- qualité de la soutenance finale
- pertinence des choix techniques

Vous n'oublierez pas d'inclure les slides de votre soutenance finale ainsi qu'un rapport PDF précisant vos choix, les problèmes techniques rencontrés et les solutions trouvées.

Un bonus pourra être attribué si votre code est suffisamment testé.

1.2 Utilisation de git

Afin d'évaluer votre réalisation, en plus des rendus sur la plateforme MyGES, vous mettrez à disposition un dépôt git.

Les différentes étapes du projet devront y être clairement distinguables, que ce soit avec des tags ou des branches.

Chaque auteur de commit devra également être facilement identifiable.

1.3 Utilisation de l'IA générative

L'objectif principal de ce projet est de prendre conscience de l'impact que peuvent avoir nos choix d'implémentations. C'est donc l'occasion d'expérimenter et d'apprendre d'erreurs n'impactant pas un projet « réel ».

Il est donc très fortement recommandé de réaliser ce projet par vous-même et de ne pas vous le déléguer complètement à une IA.

Il est bien entendu possible d'utiliser une IA comme outil d'accompagnement et de facilitation, mais pas comme substitut aux réflexions.

Vous devez être en mesure d'expliquer chacun de vos choix et d'intervenir en cas de besoin sur votre code : il est important de montrer que vous savez utiliser l'IA, mais surtout que c'est un outil qui vous permet de faire mieux ou plus vite mais pas votre remplaçant.

2 Déroulement du projet

La réalisation du projet sera découpée en trois phases :

1. Implémentation naïve d'une solution
2. Mise en place de design patterns
3. Implémentation de modules supplémentaires "libres"

Les détails de chacune des phases vous seront présentés progressivement au fil des séances. Cela permettra de vous confronter directement à vos choix d'implémentation et vous devrez donc vous demander quelles solutions restent pertinentes ou non.

2.1 Implémentation naïve d'une solution

Dans un premier temps vous devez faire une l'implémentation naïve du sujet.

Il vous est demandé de répondre aux contraintes présentées dans la suite de ce document sans réfléchir particulièrement aux design patterns qui pourraient s'y appliquer, cela se fera dans les itérations suivantes.

Cette étape doit être vue comme la mise en place d'un proof of concept et non comme la réalisation d'un produit fini.

2.2 Mise en place de design patterns

Dans un second temps, vous devrez mettre en place les design patterns que vous jugerez adaptés.

Pour cela, vous avez deux possibilités :

- Repartir du code précédent : vous aurez alors un aperçu des modifications pouvant être nécessaires sur une base de code existante dans le cadre d'un tel refacto
- Repartir d'une feuille blanche : avoir effectué une première implémentation et rencontré certaines difficultés non prévues vous permet d'avoir une vision plus claire des problématiques à gérer

Bien que les deux possibilités puissent se présenter dans des plus gros projets, vous serez plus souvent amenés à modifier du code déjà existant. Il est donc recommandé de repartir de votre code précédent.

Aucune différence de notation ne sera faite sur ce choix.

Un premier module supplémentaire vous sera également demandé lors de cette seconde phase du projet.

2.3 Implémentation de modules supplémentaires « libres »

Dans cette dernière phase du projet vous devrez choisir au moins deux modules à ajouter à votre solution.

Pour cela, divers modules vous seront présentés, plus ou moins propices à la mise en place de design patterns.

Il est possible que certains modules soient plus ou moins faciles à ajouter selon vos choix lors de la phase précédente du projet.

2.4 Soutenance

Une soutenance viendra conclure votre projet.

Il sera alors important d'y présenter vos choix : vous devrez non seulement mettre en avant les choix de design pattern que vous avez fait et les justifier, mais également ceux que vous avez écarté.

L'important n'est pas d'avoir une solution parfaite, mais d'être en mesure de montrer les réflexions que vous avez eu et qui ont amené au résultat final de votre projet. Toute solution écartée peut donc être intéressante à mentionner.

Lors de cette soutenance, vous devrez donc :

- Présenter vos choix de design patterns et modules, en les justifiant
- Faire une démonstration de votre résultat final
- Mettre en avant les différentes étapes et problématiques auxquelles vous avez été confronté

Différents critères seront pris en compte lors de la notation :

- La clarté et maîtrise de votre discours et démonstration
- Le respect des contraintes du sujet
- Le bon fonctionnement de votre solution
- La pertinence de vos choix de design patterns

Un bonus pourra être attribué en cas de code respectant les bonnes pratiques de développement.

3 Sujet

Une usine de drones souhaite automatiser et simplifier le suivi de sa production.

Les différentes pièces nécessaires à l'assemblage d'un drone sont produites dans d'autres usines et leur production ne fait pas partie de vos préoccupations, vous devrez uniquement vous charger du système d'assemblage final.

Vous devrez cependant prendre en considération le stock de pièces disponible.

Chaque drone est constitué de :

- 1 coque
- 1 module principal
- 1 générateur
- 1 module de déplacement
- 1 module de contrôle

Chaque drone aura également un système installé sur son module principal.

Afin de mener à bien l'assemblage d'un drone, il faudra découper le processus d'assemblage en plusieurs instructions.

On souhaite donc écrire un programme qui prend en entrée une commande de production et renvoie la liste des instructions à exécuter pour assembler les drones demandés.

3.1 Prise en compte des commandes

Pour certaines instructions de l'utilisateur, votre programme devra récupérer des commandes sous la forme d'entrée textuelle en console.

On appelle commande, un listing quantifié de drones.

Les instructions complètes sont de la forme :

```
[USER_INSTRUCTION] A Drone1
```

Où A est la quantité de Drone1.

```
[USER_INSTRUCTION] A Drone1, B Drone2, C Drone3
```

Où A, B et C sont respectivement les quantités de Drone1, Drone2 ou Drone3 à produire.

Une commande peut contenir plusieurs occurrences du même drone, ainsi une instruction de la forme :

```
[USER_INSTRUCTION] A Drone1, B Drone2, C Drone1
```

devra être considérée comme :

```
[USER_INSTRUCTION] A+C Drone1, B Drone2
```

Afin de simplifier les notations, ces commandes, servant d'arguments aux instructions seront abrégées en ARGS dans la suite du sujet.

3.2 Listing des instructions

3.2.1 Edition de l'inventaire des pièces disponibles

L'utilisateur peut demander l'inventaire des pièces disponibles avec l'instruction :

```
STOCKS
```

Vous devez pouvoir produire une sortie console indiquant l'intégralité du stock de l'usine. Cet inventaire devra inclure les différents drones produits ainsi que l'ensemble des pièces sous le format suivant :

```
A Drone1  
[...]  
B DroneN  
C Piece1  
[...]  
D PieceM
```

Où A, B, C et D représentent respectivement les quantités disponibles de Drone1, DroneN, Piece1 et PieceN.

3.2.2 Edition de l'inventaire des pièces nécessaires

L'utilisateur peut demander l'inventaire des pièces nécessaires à la production d'une commande.

Pour cela, il peut utiliser l'instruction :

```
NEEDED_STOCKS ARGS
```

Après avoir interprété l'instruction en entrée, vous devez pouvoir produire une sortie console respectant le format suivant :

```
A Drone1 :  
Quantité1 Piece1  
[...]  
QuantitéN PieceN  
B Drone2 :  
Quantité1 Piece1  
[...]  
QuantitéN PieceN  
Total :  
SommeDesQuantités1 Piece1  
[...]  
SommeDesQuantitésN PieceN
```

3.2.3 Édition des instructions d'assemblage

L'utilisateur peut demander la liste d'instructions à exécuter pour l'assemblage d'une commande donnée.

Pour cela il peut utiliser l'instruction suivante :

```
INSTRUCTIONS ARGS
```

Après avoir interprété l'instruction en entrée, vous devez pouvoir produire une sortie console contenant l'intégralité des instructions à effectuer respectant le format suivant :

```
PRODUCING Drone1
INSTRUCTION1_1 ARGS1_1
[...]
INSTRUCTION1_N ARGS1_N
FINISHED Drone1
PRODUCING Drone2
INSTRUCTION2_1 ARGS2_1
[...]
INSTRUCTION2_M ARGS2_N
FINISHED Drone2
```

Où INSTRUCTIONX_Y représente la Yème instruction à exécuter pour la production du drone X et ARGSX_Y la liste d'arguments de l'instruction associée.

Dans le cas où vous devez produire plusieurs fois le même modèle de drone, votre sortie devra contenir plusieurs fois les instructions concernées.

Certaines pièces nécessitent l'installation de programmes afin de fonctionner, il faudra donc bien penser à inclure également ces étapes-là.

Si une pièce possède ce type de contrainte, cela sera mentionné directement dans la liste des pièces disponibles plus loin dans ce sujet.

Plusieurs instructions sont actuellement disponibles pour effectuer l'assemblage d'un drone :

- Indiquer le début de la production du drone Drone1 :

```
PRODUCING Drone1
```

- Indiquer la fin de la production du drone Drone1 :

```
FINISHED Drone1
```

- Sortir du stock A exemplaires de la pièce Piece1 :

```
GET_OUT_STOCK A Piece1
```

- Assembler les pièces Piece1 et Piece2 afin de produire Assembly1 :

```
ASSEMBLE Assembly1 Piece1 Piece2
```

Il est possible d'utiliser cette instruction sans nommer le résultat, mais cela peut complexifier la suite des instructions. Un exemple est disponible en fin de sujet mettant en avant cela.

Dans ce cas l'instruction sera de la forme :

```
ASSEMBLE Piece1 Piece2
```

- Installer le System1 sur la Piece1 :

```
INSTALL System1 Piece1
```

Certaines contraintes doivent être respectés :

- Les pièces doivent être sorties du stock avant d'être utilisées pour un assemblage ou l'installation d'un système
- Seul le générateur peut être monté dans la coque avant le module principal
- Le module de déplacement doit être assemblé après l'ajout de la coque
- Les systèmes doivent être installés sur les pièces le nécessitant avant leur assemblage

Ce jeu d'instruction peut être amené à évoluer dans la suite du projet.

Ces instructions ne sont pas disponibles pour l'utilisateur et n'existent que pour indiquer les étapes à suivre.

3.2.4 Vérification d'une commande

L'utilisateur peut demander la vérification d'une commande donnée.

Pour cela il peut utiliser l'instruction suivante :

```
VERIFY ARGS
```

Si la commande est incorrecte, le résultat sera affiché sous la forme :

```
ERROR Message
```

Où Message indique pourquoi la commande est incorrecte

Si la commande est valide et que le stock est suffisant, le résultat sera :

```
AVAILABLE
```

Si la commande est valide mais que le stock n'est pas suffisant, le résultat sera :

```
UNAVAILABLE
```

3.2.5 Production d'une commande

L'utilisateur peut demander la production d'une commande donnée.

Pour cela il peut utiliser l'instruction suivante :

```
PRODUCE ARGS
```

Si la commande est incorrecte ou ne peut pas être produite, le résultat sera affiché sous la forme :

```
ERROR Message
```

Où Message indique pourquoi la commande est incorrecte

Si la commande peut être exécutée sans souci, les stocks doivent être mis à jour et le résultat sera affiché sous la forme :

```
STOCK_UPDATED
```

Où Message indique pourquoi la commande est incorrecte

3.3 Notations spécifiques

Un assemblage non nommé aura pour notation la liste des pièces le composant, entre crochets, séparés par une virgule.

Par exemple, l'assemblage des pièces Piece1, Piece2 et Piece3 aura pour notation :

```
[Piece1, Piece2, Piece3]
```

L'ordre et les espaces dans la notation ne sont pas bloquants.

Ainsi, la notation suivante est équivalente :

```
[Piece2, Piece1 ,Piece3]
```

Une fois qu'une pièce a un système installé, elle aura pour notation le nom de la pièce suivi du nom du système entre accolade sans aucun espace.

Ainsi la notation de la pièce Piece1 suite à l'installation du système System1 deviendra :

```
Piece1{System1}
```

3.4 Comportement attendu du programme

Votre programme devra valider chacune des entrées utilisateur et ne pas crasher :

- En cas d'entrée valide, votre programme écrira dans la console la sortie attendue
- En cas d'entrée invalide, votre programme produira une erreur compréhensible

Une fois lancé, votre programme devra permettre l'exécution de plusieurs instructions à la suite sans avoir besoin d'être relancé.

4 Première extension de projet

4.1 Mise en place de design patterns

La première contrainte de cette première extension du projet est d'ajouter les design pattern que vous jugez pertinents.

Vous devez être en mesure de justifier chacun des design patterns présents dans votre projet. Vous pouvez bien entendu vous rendre compte plus tard qu'un choix fait lors de cette étape n'était finalement pas aussi positif que ce que vous pensiez initialement. Il sera important de pouvoir mettre en avant ce qui vous a fait changer d'avis.

4.2 Différents types de drones

Afin de simplifier le stockage des drones, chacun des drones produits devra être classé dans une des catégories décrites ci-dessous.

Les catégories de drones doivent respecter au moins les contraintes suivantes :

- Aérien (F)
 - Un module de déplacement (F)
 - Un système de type (3D)
- Marin (M)
 - Une coque étanche (S)
 - Un système de type (2D)
 - Un module de déplacement (M)
- Terrestre (L)
 - Un module de déplacement (L)
 - Un système de type (2D)
- Submersible (S)
 - Toutes les pièces sont de type (S)
 - Un système de type (3D)

Rien n'empêche un drone d'appartenir à plusieurs catégories. Par exemple un drone avec une coque étanche et un module de déplacement à la fois (M) et (L) sera à la fois Terrestre et Marin. Un drone est cependant obligé d'appartenir à une de ces catégories ! Il ne peut pas y avoir de drone sans catégorie.

Les modules principaux et de contrôle ne sont pas limitants dans les contraintes de catégorisation des drones. Cependant, un module principal ne permettra pas forcément l'installation de tous les systèmes.

De même, un module de contrôle doit être compatible avec le système installé sur le module principal.

Les listes des pièces et drones ont été mises à jour pour ajouter ces précisions ainsi que de nouvelles pièces.

4.3 Ajout de templates de drones

L'utilisateur peut demander l'ajout de la gestion d'un nouveau template de drone.

Pour cela il peut utiliser l'instruction suivante :

```
ADD_TEMPLATE TEMPLATE_NAME, Piece1, [...], PieceN
```

Celle-ci prend pour argument le nom du nouveau template suivi de la liste des pièces le constituant.

L'ajout d'un template est soumis à la validation des catégories du drone concerné.

Ainsi, une tentative d'ajout ne les respectant pas devra renvoyer une erreur claire et précise.

Chaque template ajouté à l'aide de cette instruction devra être utilisable dans les arguments de toutes les instructions déjà existantes.

5 Modules complémentaires

5.1 Consignes générales

Le listing des modules complémentaires est disponible sur la plateforme MyGes.

Chaque groupe doit ajouter à minima deux modules à leur implémentation.

Il est possible de proposer des modules différents qui pourront être pris en compte dans le troisième rendu sous conditions de validation.

5.1.1 Recevoir du stock

Le programme devra être en mesure de gérer des entrées en stock.

Pour cela, vous devriez interpréter l'instruction :

```
RECEIVE ARGS
```

Où ARGS est la liste de pièces, assemblages ou drones à ajouter au stock.

5.1.2 Contraintes des constructions de drones

Chaque drone peut maintenant avoir jusqu'à 3 modules de déplacement et 2 générateurs.

Si un drone possède au moins 2 modules de déplacement, il devra également posséder 2 générateurs, sans quoi son template est invalide.

Ces nouvelles règles sont à prendre en compte dans toutes les instructions déjà existantes.

5.2 Détails des nouvelles instructions disponibles

Chacune des nouvelles instructions devra, de la même façon que celles déjà supportées renvoyer une erreur claire et précise en cas de besoin.

5.2.1 Modification de drones

Toutes les commandes existantes doivent pouvoir valider et prendre en compte de nouveaux formats possibles pour les drones composant une liste d'argument ARGS.

Le format initial doit bien entendu toujours être accepté.

Les nouvelles précisions possibles sont :

- Ajout de pièces :

```
A Drone1 WITH B Piece1, C Piece2
```

Cette précision devra ajouter B exemplaires de la pièce Piece1 et C exemplaires de la pièce Piece2

- Suppression de pièces :

```
A Drone1 WITHOUT B Piece1, C Piece2
```

Cette précision devra retirer B exemplaires de la pièce Piece1 et C exemplaires de la pièce Piece2

- Remplacement de pièces :

```
A Drone1 REPLACE B Piece1, C Piece2
```

Cette précision devra remplacer B exemplaires de la pièce Piece1 par C exemplaire de la pièce Piece2

Ces différentes précisions peuvent être composées et le résultat final doit respecter les contraintes de construction.

En cas de présence de l'une des instructions de modification de drone le séparateur utilisé pour lister les drones devra devenir ;.

Ainsi, une liste d'arguments de la forme suivante doit pouvoir être traitée par le programme :

```
A Drone1 REPLACE B Piece1, C Piece2 WITH D Piece3; E Drone2; F Drone3  
WITHOUT G Piece4
```

Attention à ce que votre programme continue bien de pouvoir traiter le format de commande initial sans précision et utilisant la virgule comme séparateur !

5.2.2 Gestion de commandes

L'utilisateur doit pouvoir passer des commandes de drones.

Dans ce cas, il passera commande avec l'instruction :

```
ORDER ARGS
```

Si l'instruction est valide, elle devra renvoyer un identifiant unique de commande (remplacé par ORDERID dans la suite), sinon elle indiquera clairement l'erreur rencontrée.

Il faudra ensuite envoyer (sortir du stock) les drones de cette commande avec l'instruction :

```
SEND ORDERID, ARGS
```

S'il reste encore des drones à envoyer pour satisfaire la commande, alors le retour de l'instruction sera de la forme :

```
Remaining for ORDERID : ARGS
```

Avec ARGS le détail des drones restants.

Si la commande est alors satisfaite, le retour sera de la forme :

```
COMPLETED ORDERID
```

On pourra, à tout moment, demander le listing des commandes restantes avec l'instruction :

```
LIST_ORDER
```

Le retour de cette instruction sera de la forme :

```
ORDERID1: ARGS1  
[...]
```

```
ORDERID2: ARGS2
```

Avec ARGSX le détail des drones restants à préparer pour la commande ORDERIDX.

5.2.3 Traçabilité des flux

L'utilisateur doit pouvoir consulter l'historique des mouvements de stock.

Pour cela, il utilisera l'instruction suivante :

```
GET_MOVEMENTS
```

Cette instruction renverra l'intégralité des instructions ayant eu un impact sur le stock.

Il devra également être possible de consulter l'historique des mouvements concernant un ou plusieurs éléments stockables (pièce, assemblage ou drone) :

```
GET_MOVEMENTS ARGS
```

Où ARGS représente la liste des éléments dont nous souhaitons consulter l'historique.

5.2.4 Gestion d'un groupe d'usines

Le programme doit être en mesure de gérer plusieurs usines.

L'utilisateur doit notamment pouvoir transférer du stock entre plusieurs usines.

Pour cela, il utilisera l'instruction suivante :

```
TRANSFER Usine1, Usine2, ARGS
```

Où Usine1 désigne l'usine de départ, Usine2 l'usine d'arrivée et ARGS la liste de stock à transférer

Il devra également pouvoir préciser l'usine de destination des instructions déjà existantes avec la précision :

```
IN Usine1
```

Où Usine1 désigne l'usine cible.

Cette précision concerne toutes les instructions ayant un impact sur les stocks ou les templates. Par exemple, l'instruction GET_STOCKS pourra être utilisé avec la précision, renvoyant alors le stock d'une usine, ou sans la précision, renvoyant alors l'intégralité du stock de toutes les usines confondues.

Une instruction de production aura maintenant la forme :

```
PRODUCE ARGS IN Usine1
```

Si l'utilisateur ne précise pas d'usine cible, il faudra alors clairement lui indiquer l'argument manquant et lui lister les usines dans lesquelles l'opération est valide.

Par exemple, si trois usines Usine1, Usine2 et Usine3 sont disponibles mais que seules Usine1 et Usine3 ont le stock nécessaire pour produire 1 DXF-1 et que l'utilisateur rentre la commande :

```
PRODUCE 1 DXF-1
```

Il aura alors comme réponse :

```
ERROR Missing target factory. Available factory for this instruction are  
Usine1 and Usine3
```


6 Récapitulatifs et données

6.1 Récapitulatif des instructions disponibles pour l'utilisateur

Cinq instructions sont actuellement disponibles pour l'utilisateur :

- Demander le niveau de stock :

```
STOCKS
```

- Demander le stock nécessaire à la production de la commande ARGS :

```
NEEDED_STOCKS ARGS
```

- Demander la liste d'instructions nécessaire à la production de la commande ARGS :

```
INSTRUCTIONS ARGS
```

- Vérifier la commande ARGS :

```
VERIFY ARGS
```

- Produire la commande ARGS :

```
PRODUCE ARGS
```

- Ajouter un template TEMPLATE_NAME contenant les pièces 1 à N :

```
ADD_TEMPLATE TEMPLATE_NAME, Piece1, [...], PieceN
```

Ce jeu d'instruction peut être amené à évoluer dans la suite du projet.

6.2 Pièces et drones disponibles

Les pièces utilisées par l'usine sont :

- Coques :
 - Hull_HG1 – (S)
 - Hull_HF1
 - Hull_HS1 – (S)
- Modules principaux (nécessitent l'installation d'un système principal) :
 - Core_CG1 – (2D)
 - Core_C3D1 – (2D) (3D)
- Générateurs :
 - Generator_GG1
 - Generator_GF1
 - Generator_GS1 – (S)

- Modules de déplacement :
 - Move_MF1 – (F)
 - Move_ML1 – (L)
 - Move_MS1 – (S)
 - Move_MM1 – (M)
 - Move_MU1 – (M) (L)
 - Move_MS2 – (M) (S)
- Modules de contrôle :
 - Processor_PG1 - (2D)
 - Processor_P3D1 – (3D)
 - Processor_PU1 – (2D) (3D)

Les systèmes principaux disponibles sont :

- System_SG1 – (2D)
- System_S3D1 – (2D) (3D)

L'usine est actuellement capable de produire une variété relativement limitée de drones :

- DXF-1 (F)
 - Hull_HF1
 - Core_C3D1 (System_S3D1)
 - Generator_GF1
 - Move_MF1
 - Processor_P3D1
- RDL-1 (L)
 - Hull_HG1
 - Core_CG1 (System_SG1)
 - Generator_GG1
 - Move_ML1
 - Processor_PG1
- WDS-1 (S)
 - Hull_HS1
 - Core_C3D1 (System_S3D1)
 - Generator_GS1
 - Move_MS1
 - Processor_P3D1
- DYM-1 (M)
 - Hull_HG1
 - Core_CG1 (System_SG1)
 - Generator_GG1
 - Move_MM1
 - Processor_PG1

7 Exemples

7.1 Demande d'instructions valide

Entrée utilisateur :

```
INSTRUCTIONS 1 DXF-1
```

Sortie :

```
PRODUCING DXF-1
GET_OUT_STOCK 1 Hull_HF1
GET_OUT_STOCK 1 Core_C3D1
GET_OUT_STOCK 1 Generator_GF1
GET_OUT_STOCK 1 Move_MF1
GET_OUT_STOCK 1 Processor_P3D1
INSTALL System_S3D1 Core_C3D1
ASSEMBLE TMP1 Hull_HF1 Generator_GF1
ASSEMBLE TMP2 TMP1 Move_MF1
ASSEMBLE TMP2 Core_C3D1{System_S3D1}
ASSEMBLE [TMP2, Core_C3D1{System_S3D1}] Processor_P3D1
FINISHED DXF-1
```

Explication détaillée des lignes d'assemblage :

On nomme l'assemblage (Hull_HF1 + Generator_GF1) TMP1 afin de pouvoir le réutiliser dans l'assemblage suivant :

```
ASSEMBLE TMP1 Hull_HF1 Generator_GF1
```

On ne nomme pas le résultat du premier assemblage ce qui nous contraint à indiquer sa composition complète :

```
ASSEMBLE TMP2 Core_C3D1{System_S3D1}
ASSEMBLE [TMP2, Core_C3D1{System_S3D1}] Processor_P3D1
```

Enfin, il n'est pas nécessaire de nommer le dernier assemblage car il s'agit d'un assemblage connu, le DXF-1.

```
ASSEMBLE [TMP2, Core_C3D1{System_S3D1}] Processor_P3D1
FINISHED DXF-1
```

7.2 Vérification d'une commande invalide

Entrée utilisateur :

```
VERIFY 1 DXF-1, 1 Cat
```

Sortie :

```
ERROR `Cat` is not a recognized drone
```